RewardCrowd — Technical Project Documentation

Team Members: Muratov Nurym, Dmukhailo Darya, Zarina Kossaman

Github link: https://github.com/fawaqw/Crowdfunding

1. Architecture Overview

This project is a small crowdfunding decentralized application (dApp) that runs on a local Ethereum environment using Hardhat.

It has three parts:

1. Smart contracts (Solidity + Hardhat)

- Crowdfunding contract: the main contract that stores campaigns, accepts ETH contributions, and mints reward tokens.

- RewardToken contract: a basic ERC-20 token used as a reward for contributors.

2. Frontend (HTML/CSS)

- The frontend is a static website built without frameworks. It connects to MetaMask and uses ethers.js v6 to call smart contract functions.

The UI supports:

- connecting a wallet,
- creating campaigns,
- contributing ETH,
- finalizing campaigns,
- showing ETH and reward token balances.

3. Local blockchain / execution environment

- The project is intended to run on a Hardhat local network:

- chainId: 31337
- RPC: http://127.0.0.1:8545

- The frontend uses the deployed contract address to interact with the Crowdfunding contract.

How data flows

1. User clicks Connect Wallet - MetaMask provides an account.
2. Frontend creates an ethers BrowserProvider and Signer.
3. Frontend connects to Crowdfunding using its address + ABI.
4. Frontend queries rewardToken() from Crowdfunding -> gets the ERC-20 token address.
5. Frontend connects to RewardToken (ERC-20 ABI) to show token balance and token.

2. Design and Implementation Decisions

The frontend was intentionally kept simple and lightweight. It is built with plain HTML, CSS, and JavaScript (no frameworks), because the goal of the project is to demonstrate blockchain interaction rather than complex UI architecture.

Key design choices:

- A clean layout with a modern “card” interface.
- A consistent green accent color used for highlights and buttons.
- Smooth transitions and spacing to make the UI easy to read.
- Clear separation of actions: creating campaigns, viewing campaigns, contributing, and finalizing.

In the JavaScript code, the main blockchain actions were separated into different functions (connect wallet, create campaign, contribute, finalize, load campaigns). This makes the code easier to understand and easier to debug.

Why two contracts instead of one ?

The system uses two separate contracts: Crowdfunding and RewardToken.

RewardToken was separated from Crowdfunding for several reasons:

- it keeps responsibilities separated (crowdfunding logic vs token logic),
- it demonstrates how an ERC-20 token is typically implemented,
- it allows controlled minting: only the Crowdfunding contract is allowed to mint reward tokens.

This design also matches common real-world patterns where tokens and application logic are deployed as separate contracts.

3. Smart Contract Logic

3.1 RewardToken.sol

Purpose

RewardToken is a custom ERC-20 token that is minted automatically as a reward when users contribute to crowdfunding campaigns.
It has no real monetary value and is included mainly for educational purposes.

State variable

```
address public crowdfundingContract;
```

This variable stores the address of the Crowdfunding contract that is allowed to mint reward tokens. Only that contract is authorized.

Constructor

```
constructor() ERC20("Reward", "RWRD") Ownable(msg.sender) {}
```

This initializes the ERC-20 token with:

- Name: Reward
- Symbol: RWRD

No tokens are minted at deployment time.

Main functions

setCrowdfundingContract(address _addr)

- Called once after deployment.
- Assigns the official Crowdfunding contract address.
- Restricted with onlyOwner.
- Designed so the minter cannot be changed later (for security and clarity).

mint(address to, uint amount)

- Mints tokens to a given address.
- Protected by:

```
require(msg.sender == crowdfundingContract, "Not allowed");
```

This ensures contributors cannot mint tokens themselves.

Summary for Reward Token

RewardToken is a minimal ERC-20 contract with controlled minting. The design ensures that reward distribution is fully controlled by the crowdfunding logic.

3.2 Crowdfunding.sol

Purpose

Crowdfunding is a decentralized crowdfunding contract that allows users to:

- create campaigns,
- contribute ETH,
- track contributions per user,
- finalize campaigns,
- receive reward tokens for participating.

Main state variables

```
RewardToken public rewardToken;
```

Stores the RewardToken contract instance used for minting rewards.

Campaign structure:

```
//campaign structure
struct Campaign {
    string title;
    uint goal;
    uint deadline;
    uint amountRaised;
    address creator;
    bool finalized;
}
```

Each campaign contains:

- title: name of the campaign
- goal: target funding amount (in wei)
- deadline: unix timestamp when campaign ends
- amountRaised: total ETH contributed
- creator: campaign creator address
- finalized: prevents further contributions when true

Campaigns are stored in:

```
Campaign[] public campaigns;
```

Campaign ID = index in the array.

```
// mapping from campaign ID
mapping(uint => mapping(address => uint)) public contributions;
```

This mapping stores how much each address contributed to each campaign

Events

```
//events for frontend and tracking
event CampaignCreated(uint id, string title, uint goal, uint deadline, address creator);
event Contributed(uint id, address contributor, uint amount);
event CampaignFinalized(uint id, uint amountRaised);
```

The contract emits events for easier tracking and frontend updates:

- CampaignCreated(...)
- Contributed(...)
- CampaignFinalized(...)

Events are useful because they allow off-chain services (or future frontend improvements) to track actions without constantly scanning contract storage.

Constructor

```
// link to the deployed RewardToken contract
constructor(address _addr) {
    rewardToken = RewardToken(_addr);
}
```

This links the RewardToken contract to Crowdfunding at deployment time.

Core functions

createCampaign(string _title, uint _goal, uint _duration)

```
// create a new crowdfunding campaign
function createCampaign(string memory _title, uint _goal, uint _duration) public{
    Campaign memory newCampaign = Campaign({
        title: _title,
        goal: _goal,
        deadline: block.timestamp + _duration,
        amountRaised: 0,
        creator: msg.sender,
        finalized: false
    });
    campaigns.push(newCampaign);
    // emit event for frontend tracking
    emit CampaignCreated(campaigns.length - 1, _title, _goal, block.timestamp + _duration, msg.
}
```

Creates a new campaign.

Main logic:

- Any user can create a campaign.

- The contract sets:
    - deadline = block.timestamp + _duration
    - amountRaised = 0
    - finalized = false
- Emits CampaignCreated.

contribute(uint _id) payable

```
// contribute ETH to an active campaign
function contribute(uint _id) public payable {
    Campaign storage campaign = campaigns[_id];
    require(!campaign.finalized, "Campaign is finalized");

    require(campaign.deadline > block.timestamp, "Campaign has ended");
    require(msg.value > 0, "You must contribute at least 1 wei");

    campaign.amountRaised += msg.value;

    contributions[_id][msg.sender] += msg.value;

    rewardToken.mint(msg.sender, msg.value * 100);

    emit Contributed(_id, msg.sender, msg.value);

}
```

Allows users to contribute ETH to a campaign.

Checks:

- Campaign must not be finalized.
- Current time must be before the deadline.
- Contribution must be greater than 0.

Effects:

- Adds to campaign.amountRaised.
- Updates contributions[_id][msg.sender].

Reward logic:

- Mints reward tokens to the contributor:

```
rewardToken.mint(msg.sender, msg.value * 100);
```

This means reward tokens are proportional to the amount of ETH contributed.

finalizeCampaign(uint _id)

Finalizes a campaign and prevents further contributions.

```
// finalize a campaign
function finalizeCampaign(uint _id) public {
Campaign storage campaign = campaigns[_id];

require(!campaign.finalized, "Campaign already finalized");

require(
    block.timestamp >= campaign.deadline || msg.sender == campaign.creator,
    "Only creator can finalize before deadline"
);

campaign.finalized = true;

emit CampaignFinalized(_id, campaign.amountRaised);
}
```

Rules:

- Campaign cannot already be finalized.
- Finalization is allowed when:
    - the deadline has passed, OR
    - the sender is the campaign creator

When finalized:

- finalized = true
- Emits CampaignFinalized.

getCampaign(uint _id)

```
// get all details of a campaign
function getCampaign(uint _id) public view returns (
    string memory,
    uint,
    uint,
    uint,
    address,
    bool
    ) {
        Campaign storage c = campaigns[_id];
        return (
            c.title,
            c.goal,
            c.deadline,
            c.amountRaised,
            c.creator,
            c.finalized
            );
}
```

Returns all campaign details in a single call:

- title, goal, deadline, amountRaised, creator, finalized

This is used by the frontend to display campaigns.

getRewardBalance(address user)

```
// get reward token balance of a user
function getRewardBalance(address user) public view returns (uint) {
return rewardToken.balanceOf(user);
```

Returns the RewardToken balance of a given address.
This function is optional but helps the frontend show token balances easily.

4. Frontend-to-Blockchain Interaction

The frontend is located in frontend/ and includes:

- index.html
- style.css
- app.js

The frontend uses ethers.js and MetaMask to interact with the contracts.

Wallet connection

When the user clicks “Connect Wallet”, the app:

1. Detects MetaMask (window.ethereum)

```
async function connectWallet() {
  try {
    if (!window.ethereum) {
      alert("MetaMask not found. Install MetaMask and try again.");
      return;
    }
```

2. Creates a provider:

```
provider = new ethers.BrowserProvider(window.ethereum);
```

3. Requests accounts:

```
// request accounts
await provider.send("eth_requestAccounts", []);
signer = await provider.getSigner();
user = await signer.getAddress();
```

4. Gets the signer and wallet address.

```
signer = await provider.getSigner();
user = await signer.getAddress();
```

5. Checks the network chainId (expects 31337).

```
// network
const net = await provider.getNetwork();
uiNetwork.textContent = "Network: " + net.chainId;
```

```
if (net.chainId !== 31337n) {
  // user must switch network
  alert("Please switch MetaMask network to Hardhat Local (chainId 31337).");
  // still continue to show address
}
```

The app also attaches listeners so the page reloads when the user changes account or network:

```
// attach listeners
if (window.ethereum && window.ethereum.on) {
  window.ethereum.on("accountsChanged", () => location.reload());
  window.ethereum.on("chainChanged", () => location.reload());
}
```

Contract instantiation

After connecting the wallet, the frontend creates an ethers Contract instance for the Crowdfunding contract using:

- the Crowdfunding ABI
- the signer (so write transactions can be signed
- Crowdfunding adress

```
crowdfunding = new ethers.Contract(CROWDFUNDING_ADDRESS, crowdfundingAbi, signer);
```

Then the frontend reads the RewardToken address directly from Crowdfunding:

```
const rewardAddr = await crowdfunding.rewardToken();
```

And instantiates the RewardToken contract:

```
rewardToken = new ethers.Contract(rewardAddr, rewardAbi, signer);
```

After that, the frontend reads token metadata:

```
try {
  rewardDecimals = Number(await rewardToken.decimals());
} catch (e) {
  rewardDecimals = 18;
}
```

Reading data from the blockchain (view calls)

The frontend uses view functions to load campaigns and balances.
These calls do not create transactions and do not cost gas.

ETH balance

ETH balance is loaded using the provider:

```
async function refreshBalances() {
  try {
    if (!provider || !user) return;
    const eth = await provider.getBalance(user);
    uiEthBalance.textContent = Number(formatEth(eth)).toPrecision(6).replace(/\.?0+$/, "");
    if (rewardToken) {
      const rbal = await rewardToken.balanceOf(user);
      uiRwrdBalance.textContent = Number(formatToken(rbal, rewardDecimals)).toString();
    }
    showStatus("Balances updated", 1500);
  } catch (err) {
    console.error(err);
    showStatus("Balances update failed");
  }
}
```

Reward token balance

RewardToken balance is loaded from the ERC-20 contract:

```
async function refreshBalances() {
  try {
    if (!provider || !user) return;
    const eth = await provider.getBalance(user);
    uiEthBalance.textContent = Number(formatEth(eth)).toPrecision(6).replace(/\.?0+$/, "");
    if (rewardToken) {
      const rbal = await rewardToken.balanceOf(user);
      uiRwrdBalance.textContent = Number(formatToken(rbal, rewardDecimals)).toString();
    }
    showStatus("Balances updated", 1500);
  } catch (err) {
    console.error(err);
    showStatus("Balances update failed");
  }
}
```

Loading and displaying campaigns

To load campaigns, the app first reads the number of campaigns:

```
async function loadCampaigns() {
  try {
    if (!crowdfunding) { uiCampaigns.innerHTML = "<p>Connect wallet to load campaigns</p>"; return; }
    uiCampaigns.innerHTML = "<p>Loading campaigns...</p>";

    const countBig = await crowdfunding.getCampaignCount();
    const count = Number(countBig);

    if (count === 0) {
      uiCampaigns.innerHTML = "<p>No campaigns yet</p>";
      return;
    }

    uiCampaigns.innerHTML = "";
```

Then it loops through each campaign and reads campaign data:

```
for (let i = 0; i < count; i++) {
  const raw = await crowdfunding.getCampaign(i);
  const parsed = await parseCampaignStruct(raw, i);
```

The campaign is returned as a tuple:

- title
- goal
- deadline
- amountRaised
- creator
- finalized

The frontend parses it into a readable object and calculates progress:

```
// helper to parse returned campaign tuple
async function parseCampaignStruct(raw, id) {
  const title = raw[0];
  const goalWei = raw[1];
  const deadlineUnix = Number(raw[2]);
  const raisedWei = raw[3];
  const creator = raw[4];
  const finalized = raw[5];

  const goalEth = Number(formatEth(goalWei));
  const raisedEth = Number(formatEth(raisedWei));

  // percent safe calculation
  const percent = (goalEth === 0 ? 0 : Math.min((raisedEth / goalEth) * 100, 100));

  // user contribution
  let yourContribution = 0;
  try {
    const contrib = await crowdfunding.contributions(id, user);
    yourContribution = Number(formatEth(contrib));
  } catch (e) {
    // ignore
  }
```

Contribution tracking per user

The app also shows the connected user’s contribution for each campaign using:

```
try {
  const contrib = await crowdfunding.contributions(id, user);
  yourContribution = Number(formatEth(contrib));
} catch (e) {
  // ignore
}
```

This value is displayed on each campaign card as:

- “Your contribution: X ETH”

Writing transactions (state-changing calls)

Unlike view calls, the following functions send blockchain transactions:

- createCampaign(...)
- contribute(...)
- finalizeCampaign(...)

Transactions require:

- MetaMask confirmation
- gas fees (on local Hardhat chain this is test ETH)
- waiting for mining using tx.wait()

Creating a campaign

The frontend collects inputs from the form and converts them into contract values:

- goal ETH → wei:
- duration minutes → seconds:

```
const goalWei = ethers.parseEther(goalStr);
const durationSec = Math.floor(Number(durationStr) * 60);
```

Then it sends the transaction:

```
showStatus("Sending transaction...");
const tx = await crowdfunding.createCampaign(title, goalWei, durationSec);
await tx.wait();
```

Contributing to a campaign (payable)

Contributions send ETH to the contract by attaching value:

```
async function contributeToCampaign(id, amountEth) {
  try {
    if (!crowdfunding) return alert("Connect wallet first");
    const value = ethers.parseEther(String(amountEth));
    showStatus("Sending contribution tx...");
    const tx = await crowdfunding.contribute(id, { value });
    await tx.wait();
    showStatus("Contribution confirmed", 3000);
    await refreshBalances();
    await loadCampaigns();
  } catch (err) {
    console.error("contribute err", err);
    showStatus("Contribution failed: " + (err?.message || err), 5000);
  }
}
```

This becomes msg.value inside the Solidity contract.

Finalizing a campaign

Finalization is done with:

```
async function finalizeCampaign(id) {
  try {
    if (!crowdfunding) return alert("Connect wallet first");
    showStatus("Sending finalize tx...");
    const tx = await crowdfunding.finalizeCampaign(id);
    showStatus("Waiting for confirmation...");
    await tx.wait();
    showStatus("Campaign finalized", 3000);
    await loadCampaigns();
  } catch (err) {
    console.error("finalize err", err);
    const nice = parseTxError(err);
    showStatus(nice, 6000);
  }
}
```

The UI prevents invalid actions (finalized/ended) and shows a user-friendly message; the contract still enforces the rules.

Updating the UI after transactions

After every successful transaction, the frontend reloads data to stay consistent with the blockchain:

```
await refreshBalances();
await loadCampaigns();
```

This ensures:

- updated ETH balance
- updated reward token balance
- updated campaign progress
- updated finalized status

5. Deployment and Execution Instructions

This project is intended to run on a local Hardhat network and be used through MetaMask + a local web server.

5.1 Prerequisites

- Node.js installed
- npm (comes with Node.js)
- MetaMask browser extension
- Project dependencies installed via npm install

Install dependencies

npm install

installs all project dependencies listed in package.json (including Hardhat and ethers.js).

5.2 Start the Hardhat local blockchain

Open Terminal 1 and run:

npx hardhat node

Keep this terminal running. Hardhat will print:

- local RPC URL
- test accounts
- private keys
- pre-funded ETH balances

5.3 Deploy the smart contracts

Open Terminal 2 (separate terminal window) and run:

npx hardhat run scripts/deploy.js --network localhost

What this script does (in order):

1. Deploys RewardToken
2. Deploys Crowdfunding, passing the token address to the constructor:

After deployment, you will see the printed addresses:

- RewardToken address
- Crowdfunding address

Update the frontend contract address

Open frontend/app.js and replace:

const CROWDFUNDING_ADDRESS = "0xa513E6E4b8f2a923D98304ec87F64353C4D5C853";

with the newly deployed Crowdfunding address from Terminal 2

This step matters because if you redeploy, the contract address changes, and the UI must point to the correct address.

5.4 Run the frontend locally

You should run the frontend using a local server (recommended).
From the frontend/ folder:

npx serve .

Then open the URL printed in the terminal (for example http://localhost:3000 or http://127.0.0.1:5173).

5.5 Connect MetaMask to Hardhat Local Network

In MetaMask, add a new network:

- Network Name: Hardhat Local
- RPC URL: http://127.0.0.1:8545
- Chain ID: 31337
- Currency Symbol: ETH

Switch to this network before using the dApp.

5.6 Import a funded test account into MetaMask

Hardhat prints test accounts with private keys in Terminal 1.

To import:

1. Copy one private key from Hardhat output
2. MetaMask -> Account menu -> Import account
3. Paste the private key

This imported account will already have test ETH.

6. Process for Obtaining Test ETH

Because the project runs on a local Hardhat blockchain, test ETH is provided automatically. The steps for running local node is included in the 5 part.